

CS149 Assignment 2

Yifan Luo

September 12, 2026

Contents

1	Overview	1
2	Task System Implementation	1
2.1	Part A: Synchronous Bulk Task Launch	1
2.2	Part B: Supporting Execution of Task Graphs	3
3	Performance Analysis	3
3.1	When Simpler Implementations Perform Better	3
3.2	Implementation Remarks and Optimizations	4
3.3	Performance Comparison with the Reference Implementation	5
4	Custom Test	6

Environment

All experiments in this assignment were conducted on my personal computer, which is equipped with an Intel Core i9-13900HX processor. The CPU has 24 physical cores and 32 logical threads, with 8 of the cores supporting Hyper-Threading. Since the experiments were not conducted on the AWS `c7g.4xlarge` instance provided by Stanford for CS149 students, the performance characteristics and experimental results may differ from those obtained on the official course hardware. To make the experimental setup more comparable to the official AWS instance, which provides 16 execution contexts, I limited my implementations to at most 16 threads for the experiments reported in this writeup.

1 Overview

This project implements a task execution library that supports bulk task launches, similar to the task abstraction provided by ISPC.

In this write-up, we describe the design and implementation of our task system, analyze its performance under different workloads, and introduce a custom test designed to evaluate an important improvement over our initial naive implementation.

2 Task System Implementation

This section is organized into two parts. In the first part, we describe the design of our synchronous bulk task launch system, in which all tasks in a launch are guaranteed to complete before the calling thread returns. In the second part, we extend the task system to support asynchronous task launches with dependencies on previously submitted tasks.

2.1 Part A: Synchronous Bulk Task Launch

Following the provided assignment guidelines, we gradually introduced additional features into the synchronous task system.

Our first, naive implementation does not use a thread pool. Instead, a new set of worker threads is spawned for each bulk task launch and joined before the call to `run()` returns. We then introduced a

persistent thread pool, in which worker threads are created when the task system is instantiated and destroyed when the task system is destructed. In this second implementation, however, both worker threads and the calling thread use busy waiting, which can waste substantial CPU resources while no useful work is being performed. In the final implementation, worker threads are put to sleep whenever no tasks are available and are notified when new work arrives. Similarly, the calling thread sleeps while waiting for all tasks in the current bulk launch to complete.

In the remainder of this subsection, we focus only on the final implementation.

Our final implementation uses a persistent thread pool. The worker threads are created when the task system is constructed and remain alive until the task system is destroyed. Each worker repeatedly waits for available work and sleeps when no task is ready to execute.

Tasks within a bulk launch are assigned to workers dynamically. Rather than partitioning the task indices among threads in advance, all workers obtain task IDs from a shared counter. Whenever a worker finishes one task, it can immediately acquire another task if work remains. This dynamic scheduling strategy is particularly useful when tasks within the same bulk launch have different execution costs, since faster workers are not forced to remain idle after completing a fixed partition of the workload.

When `run()` is called, the task system records the current bulk launch and notifies the worker threads that new work is available. The workers then execute tasks concurrently until all tasks in the launch have finished. The calling thread does not participate in task execution; instead, it sleeps while waiting for completion and is notified when the final task finishes. This preserves the synchronous semantics of `run()`: when it returns, all tasks in the corresponding bulk launch have completed.

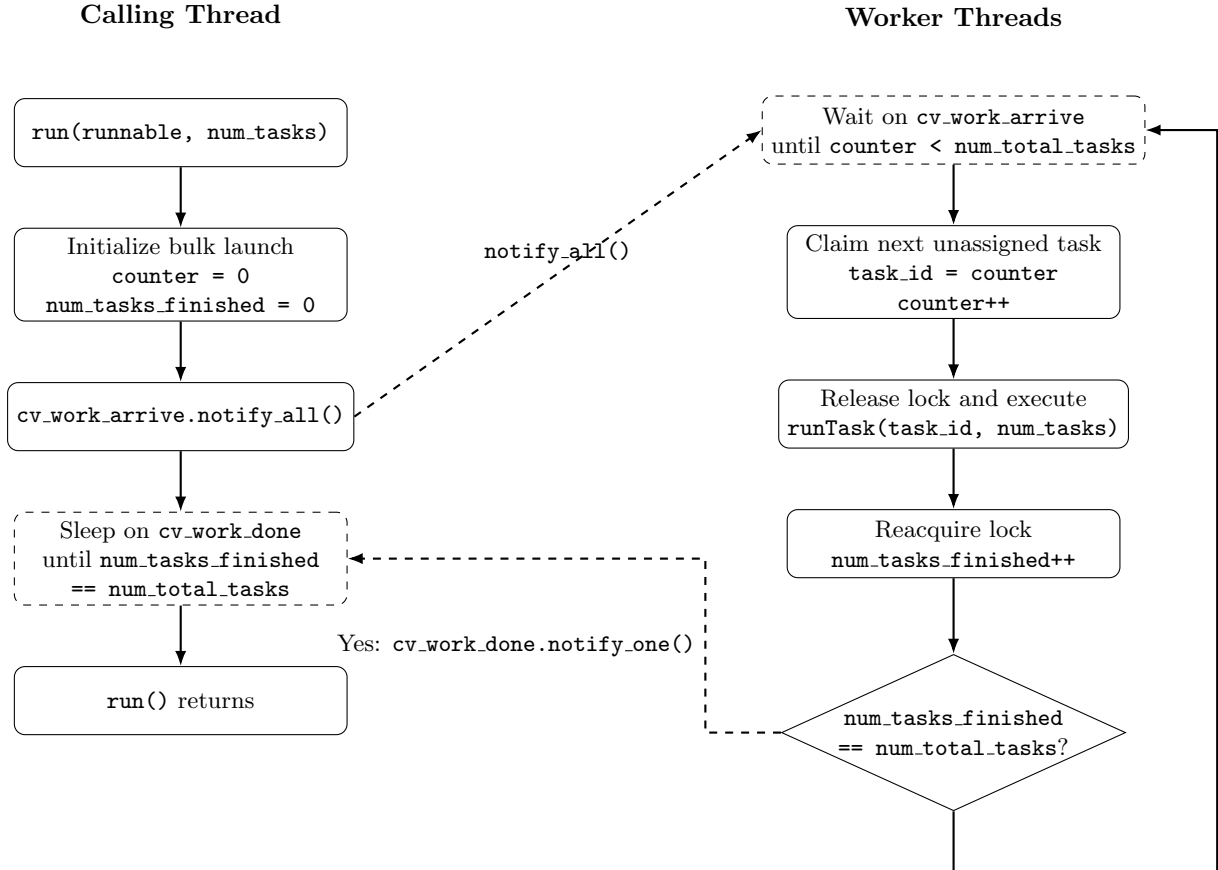


Figure 1: Synchronization between the calling thread and worker threads in the sleeping thread-pool implementation. The shared variable `counter` tracks the next unassigned task, while `num_tasks_finished` tracks tasks that have actually completed. Workers sleep when no unassigned task is available, while the calling thread sleeps until the final task completes.

2.2 Part B: Supporting Execution of Task Graphs

In Part B, we extend the task system to support dependencies between bulk launches. Each bulk launch is either waiting for its dependencies to be resolved or is placed in a ready queue once all of its dependencies have completed. The ready queue operates at the bulk-launch level: each queue entry represents an entire bulk launch containing multiple tasks. Worker threads obtain tasks from the bulk launch at the front of the ready queue.

Our initial implementation kept the front bulk launch in the ready queue until all of its tasks had finished. Although correct, this unnecessarily serialized different bulk launches: even after all tasks from the front launch had already been assigned to workers, idle workers could not begin working on the next ready launch until the current one had completely finished.

To allow parallel execution across bulk launches, we instead remove a launch from the ready queue as soon as all of its tasks have been assigned, even if some of those tasks are still running. A counter, `waiting_to_finish`, tracks the number of such launches. This allows idle workers to immediately obtain tasks from the next ready bulk launch, while `waiting_to_finish` ensures that `sync()` can still determine whether all previously submitted work has truly completed.

3 Performance Analysis

3.1 When Simpler Implementations Perform Better

More sophisticated parallel implementations do not always achieve better performance, because parallel execution introduces overhead from thread management, synchronization, and task scheduling. Whether this overhead is worth paying depends strongly on both the amount of work in each task and the number of bulk launches.

Sequential vs. Parallel. When individual tasks are extremely lightweight, the sequential implementation can outperform all parallel implementations. In this case, the useful computation is too small to amortize the overhead of creating or waking worker threads, scheduling tasks, and synchronizing execution.

For example, the results for `super_super_light` are shown in Table 1. The serial implementation completes in only 4.673 ms, substantially faster than all parallel implementations. This indicates that, for extremely lightweight tasks, the overhead introduced by parallelization can be much larger than the useful computation itself.

Table 1: Runtime of different implementations on `super_super_light`.

Implementation	Runtime (ms)
Serial	4.673
Always Spawn	256.379
Thread Pool + Spin	32.330
Thread Pool + Sleep	78.520

Always-Spawn vs. Thread Pool. The performance of the always-spawn implementation depends strongly on the number of bulk launches. When there are only a few launches, the cost of creating and joining threads is paid only a few times and can remain small relative to the useful computation.

For example, `spin_between_run_calls` contains only three bulk launches. As shown in Table 2, the always-spawn implementation takes 161.411 ms, which is almost identical to the sleeping thread-pool implementation at 160.727 ms.

Table 2: Runtime on `spin_between_run_calls`, which contains 3 bulk launches.

Implementation	Runtime (ms)
Serial	325.664
Always Spawn	161.411
Thread Pool + Spin	218.065
Thread Pool + Sleep	160.727

A similar pattern appears in `recursive_fibonacci`, which contains 30 bulk launches. The always-spawn implementation is somewhat slower than the thread-pool implementations, but the difference is still relatively small, as shown in Table 3.

Table 3: Runtime on `recursive_fibonacci`, which contains 30 bulk launches.

Implementation	Runtime (ms)
Serial	903.676
Always Spawn	90.409
Thread Pool + Spin	78.274
Thread Pool + Sleep	82.063

In contrast, repeatedly creating threads becomes very expensive when the program performs many bulk launches. The `math_operations_in_tight_for_loop` test contains 2000 bulk launches. As shown in Table 4, the always-spawn implementation takes 1288.127 ms, compared with only 179.397 ms for the spinning thread pool and 387.671 ms for the sleeping thread pool. In this case, the repeated thread creation and destruction overhead dominates the always-spawn implementation.

Table 4: Runtime on `math_operations_in_tight_for_loop`, which contains 2000 bulk launches.

Implementation	Runtime (ms)
Serial	532.708
Always Spawn	1288.127
Thread Pool + Spin	179.397
Thread Pool + Sleep	387.671

Conclusion. Overall, sequential implementations can perform well when the workload per task is too small for parallelism to amortize its overhead. Persistent thread pools become increasingly beneficial as more bulk launches reuse the same worker threads.

3.2 Implementation Remarks and Optimizations

During the implementation, we also observed several performance effects that were not immediately obvious from the high-level design of the task system.

Number of Mutexes in Part A. One interesting observation is that using more fine-grained mutexes does not necessarily result in better performance. Intuitively, separating unrelated shared states into different critical sections should reduce lock contention and allow more concurrency. This behavior was indeed observed in our spinning thread-pool implementation, where using two mutexes performed better than using a single mutex.

However, the opposite behavior was observed in the sleeping implementation: using a single mutex was noticeably faster than using two mutexes. This suggests that lock granularity alone is insufficient to predict the performance of a synchronization design. In the sleeping implementation, mutexes also interact with condition-variable wakeups and subsequent lock reacquisition, which may introduce additional synchronization overhead. We did not investigate this effect further, so its exact cause remains an interesting topic for future experiments.

Placement of Notifications in Part B. We also found that the placement of condition-variable notifications can have a surprisingly large impact on performance. In particular, notifying worker threads while holding the mutex versus after releasing it produced substantially different runtimes in some workloads.

Our final implementation uses different notification strategies in different situations. When completion of a bulk launch makes previously blocked launches ready, worker threads are notified after releasing the mutex, which avoids immediate contention between the awakened workers and the thread still holding the lock. In contrast, when the calling thread submits new ready launches, we found experimentally that notifying while still holding the mutex performs significantly better. One possible explanation is that this allows the calling thread to submit several launches with less interference from awakened worker

threads, although we have not experimentally verified this hypothesis. The result illustrates that the optimal placement of a notification depends on the surrounding workload and synchronization pattern.

Reducing Critical Sections with Atomics. Another effective optimization in Part B was to reduce the amount of work performed under the global mutex. In the initial implementation, both task-level synchronization, such as task assignment and completion counting, and bulk-launch-level operations, such as accessing and modifying the ready queue, were protected by the same mutex. We later changed the per-bulk-launch task counter and completion counter to `std::atomic<int>` variables and moved these task-level operations outside the critical section.

As a result, the global mutex is primarily used for bulk-launch-level shared state, such as the ready queue and dependency bookkeeping, while synchronization among workers executing tasks from the same bulk launch is handled using atomic operations. This reduces contention on the global mutex at the cost of additional atomic synchronization. In our experiments, the reduction in lock contention outweighed this additional cost, producing an approximately $1.2\times$ performance improvement.

3.3 Performance Comparison with the Reference Implementation

Our implementations satisfy the performance requirements of the assignment. For Part A, the runtime must be within $1.2\times$ that of the reference implementation, while Part B allows up to $1.5\times$. All of our implementations pass the provided performance tests. In most cases, the measured runtime is well within the required threshold, and several Part B tests are substantially faster than the reference implementation.

Table 5: Part A performance comparison with the reference implementation. The reported ratio is student runtime divided by reference runtime.

Test	Serial	Always Spawn	Thread Pool + Spin	Thread Pool + Sleep
<code>super_super_light</code>	0.93	0.93	0.83	0.99
<code>super_light</code>	1.01	0.93	0.82	1.04
<code>ping_pong_equal</code>	1.01	0.94	0.90	0.94
<code>ping_pong_unequal</code>	1.00	0.97	0.92	1.02
<code>recursive_fibonacci</code>	0.57	0.67	0.62	0.66
<code>math_operations_in_tight_for_loop</code>	0.98	0.95	0.97	0.99
<code>math_operations_in_tight_for_loop_fewer_tasks</code>	0.97	0.94	0.85	1.00
<code>math_operations_in_tight_for_loop_fan_in</code>	0.97	0.93	0.81	0.95
<code>math_operations_in_tight_for_loop_reduction_tree</code>	0.97	0.96	0.97	0.99
<code>spin_between_run_calls</code>	0.57	0.56	0.61	0.56
<code>mandelbrot_chunked</code>	1.00	0.77	0.98	0.97

Table 6: Part B performance comparison with the reference implementation. The reported ratio is student runtime divided by reference runtime.

Test	Ratio
<code>super_super_light_async</code>	0.13
<code>super_light_async</code>	0.59
<code>ping_pong_equal_async</code>	1.09
<code>ping_pong_unequal_async</code>	1.11
<code>recursive_fibonacci_async</code>	0.60
<code>math_operations_in_tight_for_loop_async</code>	0.86
<code>math_operations_in_tight_for_loop_fewer_tasks_async</code>	0.95
<code>math_operations_in_tight_for_loop_fan_in_async</code>	1.04
<code>math_operations_in_tight_for_loop_reduction_tree_async</code>	1.00
<code>spin_between_run_calls_async</code>	0.57
<code>mandelbrot_chunked_async</code>	1.18

The worst relative runtime in Part A remains within the required $1.2\times$ threshold, while the worst Part B result is $1.18\times$, comfortably below the allowed $1.5\times$. Several Part B workloads are significantly faster than the reference implementation, most notably `super_super_light_async`, which runs at only $0.13\times$ the reference runtime.

4 Custom Test

To evaluate the improvement described in Section 2.2, we designed a diamond-dependency test for the asynchronous task system. Our initial implementation allowed parallel execution only among tasks within the same bulk launch. After introducing `waiting_to_finish`, a bulk launch can leave the ready queue once all of its tasks have been assigned, allowing workers to begin executing tasks from later ready launches even while tasks from the previous launch are still running. The purpose of this test is to verify both the correctness of dependency resolution and whether this cross-bulk-launch parallelism is actually achieved.

The test constructs four bulk launches with the dependency graph

$$A \rightarrow \{B, C\} \rightarrow D.$$

Each bulk launch contains eight tasks, and every task simply sleeps for 100 ms. The test is run with 16 worker threads. Therefore, after *A* completes, the tasks from *B* and *C* should be able to execute in parallel. Since *B* and *C* contain a total of 16 tasks, all of them can occupy the 16 worker threads at the same time, producing approximately 100 ms of overlap if the scheduler successfully supports parallel execution across bulk launches.

The measured execution timeline is shown in Table 7.

Table 7: Timeline of the diamond-dependency test.

Launch	Start (ms)	End (ms)	Span (ms)
<i>A</i>	0.263	100.871	100.608
<i>B</i>	100.984	201.358	100.373
<i>C</i>	101.166	201.459	100.293
<i>D</i>	201.594	302.123	100.529

The observed execution order satisfies all dependency constraints: *B* and *C* begin only after *A* finishes, while *D* begins only after both *B* and *C* have completed. In addition to the timing information shown in Table 7, the test explicitly verifies that every task is executed exactly once and that `sync()` returns only after all task bodies have finished.

More importantly, the execution intervals of *B* and *C* overlap almost completely. The observed overlap is approximately 100.191 ms, which is close to the expected 100 ms. This confirms that the revised scheduler can execute different ready bulk launches in parallel rather than forcing one bulk launch to finish completely before beginning the next.

This test did not directly cause the final implementation change, since the `waiting_to_finish` optimization had already been introduced. However, it was specifically designed to validate that this optimization correctly enables parallelism across bulk launches while preserving dependency and synchronization correctness.